Session 10 - Functions, Heap, Stack, Memory Allocation

Naveen AutomationLabs | 2026 Playwright Batch G1

---

1. Functions - Basics

A function is a reusable block of code. Functions help avoid repetition and keep code organized.

Type 1: No input, no return

```
function test() {
console.log('test function');
// no return keyword = returns undefined
}

console.log(typeof test); // 'function'
test();
```

Type 2: No input, with return

```
function getTrainerName() {
let trainerName = 'naveen';
return trainerName;
}

let name = getTrainerName();
console.log(name); // naveen
```

Type 3: With input and return

```
function add(a, b) { // a, b are parameters
let c = a + b;
return c;
}

let r = add(100, 200); // 100, 200 are arguments
console.log(r); // 300
```

Parameters are names in the function definition. Arguments are actual values passed when calling the function.

2. Practical Example - getStudentMarks

```
function getStudentMarks(studentName) {
let marks = -1;

if (studentName === 'ravi') {
marks = 90;
} else if (studentName === 'uday') {
marks = 80;
} else if (studentName === 'ankit') {
marks = 100;
} else {
console.log(studentName + ' is not found');
}
```

```
return marks;
}

let m1 = getStudentMarks('ravi');
console.log(m1); // 90
```

Same logic using switch-case:

```
function getMarks(studentName) {
switch (studentName.trim().toLowerCase()) {
case 'ravi': return 90;
case 'pooja': return 80;
case 'uday': return 100;
default:
console.log(studentName + ' is not found');
return -1;
}
}
```

3. Function Properties

```
function getTotalBilling(productPrice, tax, lateFee, gst) {
return productPrice + tax + lateFee;
}

let amt = getTotalBilling(1000, 10, 20, 5);
console.log(amt); // 1030

console.log(typeof getTotalBilling); // 'function'
console.log(getTotalBilling.name); // 'getTotalBilling'
console.log(getTotalBilling.length); // 4 (number of parameters)
```

4. JavaScript Type Coercion in Functions

JavaScript does not enforce parameter types. If you pass a string where a number is expected, it will concatenate instead of adding.

```
let res = add(10, 'naveen');
console.log(res); // '10naveen' (string)
```

> Always validate your inputs if type matters. JavaScript will not throw an error for wrong types.

5. Heap and Stack Memory

JavaScript uses two types of memory:

- Stack: Stores primitive values (numbers, strings, booleans) and function call frames. Works as LIFO (Last In, First Out).
- Heap: Stores objects, arrays, and functions. These are reference types -- the variable holds a reference (pointer) to the data in heap.

When a function is called, a new frame is pushed onto the call stack. When the function finishes (returns), the frame is popped off the stack.

> If too many functions call each other without returning (infinite recursion), you get a 'Maximum call stack size exceeded' error.